

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»

Лабораторная работа №1
по курсу «Технологии параллельного программирования»

Обработка изображений на GPU. Фильтры.

Выполнил: *Серый Никита Олегович*

Группа: *M8O-409B-22*

Преподаватели: А.Ю. Морозов,
Е.Е. Заяц

Москва, 2025

Условие

1. Цель работы: Научиться использовать GPU для обработки изображений. Использование текстурной памяти и двухмерной сетки потоков.
2. Вариант 6. Метод Превитта.

Формат изображений. Изображение является бинарным файлом, со следующей структурой:

width(w)	height(h)	r	g	b	a	r	g	b	a	r	g	b	a	...	r	g	b	a	r	g	b	a
4 байта, int	4 байта, int	4 байта, значение пикселя [1,1]				4 байта, значение пикселя [2,1]				4 байта, значение пикселя [3,1]				...	4 байта, значение пикселя [w - 1, h]				4 байта, значение пикселя [w,h]			

В первых восьми байтах записывается размер изображения, далее построчно все значения пикселей, где

- r -- красная составляющая цвета пикселя
- g -- зеленая составляющая цвета пикселя
- b -- синяя составляющая цвета пикселя
- a -- значение альфа-канала пикселя

Пример картинки размером 2 на 2, синего цвета, в шестнадцатеричной записи:

```
02000000 02000000 0000FF00 0000FF00 0000FF00 0000FF00
```

Студентам предлагается самостоятельно написать конвертер на *любом* языке программирования для работы с вышеописанным форматом.

В данной лабораторной работе используются только цветовые составляющие изображения (r g b), альфа-канал не учитывается. При расчетах значений допускается ошибка в ± 1 . Ограничение: $w < 2^{16}$ и $h < 2^{16}$. Во всех вариантах, кроме 2-го и 4-го, в пограничном случае, необходимо "расширять" изображение за его границы, при этом значения соответствующих пикселей дублируют граничные. То есть, для любых индексов i и j, координаты пикселя [ip, jp] будут определяться следующим образом:

```
ip := max(min(i, h), 1)
```

```
jp := max(min(j, w), 1)
```

Входные данные. На первой строке задается путь к исходному изображению, на второй, путь к конечному изображению. $w \cdot h \leq 10^8$.

Пример:

Входной файл	hex: in.data	hex: out.data
in.data out.data	03000000 03000000 01020300 04050600 07080900 09080700 06050400 03020100 00000000 14141400 00000000	03000000 03000000 0D0D0D00 06060600 0B0B0B00 17171700 05050500 14141400 25252500 07070700 2C2C2C00

in.data out.data	03000000 03000000 00000000 00000000 00000000 00000000 80808000 00000000 00000000 00000000 00000000	03000000 03000000 B5B5B500 80808000 B5B5B500 80808000 00000000 80808000 B5B5B500 80808000 B5B5B500
---------------------	---	---

Программное и аппаратное обеспечение

Графический процессор (Google Colab)

Compute capability	7.5
Name	Tesla T4
Total Global Memory	15828320256
Shared memory per block	49152
Registers per block	65536
Warp size	32
Max threads per block	(1024, 1024, 64)
Max block	(2147483647, 65535, 65535)
Total constant memory	6553
Multiprocessors count	40

Процессор AMD Ryzen 5 5500U (2.1 GHz, max 4 GHz)

Technology	7 нм
Cores	6
Threads	12
Cache (Level 2)	64К (на ядро)
Cache (Level 2)	512К (на ядро)
Cache (Level 3)	8 МБ (всего)

Оперативная память

Number of modules	2
Module Size	8 ГБ
Frequency	3200 МГц
Generation	DDR4

Жесткий диск - 512 ГБ SSD

OS - Ubuntu 22

IDE - VS Code

Compiler - nvcc

Метод решения

Алгоритм обработки каждого пикселя (x,y) состоит из следующих этапов:

Этап А: Перевод в градиент серого

Поскольку оператор Превитта работает с интенсивностью, каждый считанный пиксель преобразуется в значение яркости по формуле: $Gray = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$.

Этап Б: Вычисление свертки (Оператор Превитта)

Для нахождения границ вычисляется градиент яркости в окрестности 3×3 . Используются два ядра (маски) для детектирования горизонтальных (Gy) и вертикальных (Gx) изменений.

Этап В: Расчет итоговой интенсивности

Амплитуда градиента вычисляется как упрощенная сумма модулей: $Val = |Gx| + |Gy|$. Полученное значение ограничивается диапазоном [0,255] и записывается в выходной массив как серый цвет.

Описание программы

Основные типы данных

uchar4: структура из четырех беззнаковых байтов (x, y, z, w) - каналы **RGBA**.

Оптимальна для выполнения 32-битные обращения к памяти на граф. процессорах.

cudaTextureObject_t: Дескриптор текстурного объекта, позволяющий ядру использовать блок текстурирования.

cudaArray: Оптимизированный под пространственные локальные данные формат памяти на GPU.

Функции выполняются на CPU:

read_data / write_data: Функции ввода-вывода. Работают с бинарным форматом, где первые 8 байт — это ширина и высота изображения (int), а далее следует массив пикселей.

create_texture: Инициализирует текстурные ресурсы: создает cudaArray, копирует данные из оперативной памяти в массив, настраивает cudaTextureDesc.

Описание ядра:

1. **Расчет координат:** Каждый поток вычисляет свои начальные координаты (x,y) на основе индексов блока и потока.
2. **Grid-Stride Loop:** Внутри ядра используется цикл по offset, что позволяет обрабатывать изображения, размеры которых превышают общее количество запущенных потоков.
3. **Выборка данных:** С помощью tex2D ядро считывает 8 соседних пикселей вокруг целевой точки.
4. **Вычисление градиента:**
 - Применяются маски Превитта для поиска горизонтальных и вертикальных перепадов яркости (Gx,Gy).
 - Находится итоговая амплитуда: $|Gx| + |Gy|$.
5. **Запись результата:** Полученное значение приводится к типу unsigned char с насыщением (максимум 255) и записывается в глобальную память.

```
__global__ void kernel(cudaTextureObject_t tex, uchar4 *out, int w, int h) {
    int idx = blockDim.x * blockIdx.x + threadIdx.x;
    int idy = blockDim.y * blockIdx.y + threadIdx.y;
    int offsetx = blockDim.x * gridDim.x;
    int offsety = blockDim.y * gridDim.y;

    float dx = 1.0f / w;
    float dy = 1.0f / h;

    for (int y = idy; y < h; y += offsety) {
        for (int x = idx; x < w; x += offsetx) {
            float fx = (float)x / w;
            float fy = (float)y / h;

            float p00 = get_gray(tex2D<uchar4>(tex, fx - dx, fy - dy));
            float p01 = get_gray(tex2D<uchar4>(tex, fx,    fy - dy));
            float p02 = get_gray(tex2D<uchar4>(tex, fx + dx, fy - dy));
            float p10 = get_gray(tex2D<uchar4>(tex, fx - dx, fy));
            float p12 = get_gray(tex2D<uchar4>(tex, fx + dx, fy));
            float p20 = get_gray(tex2D<uchar4>(tex, fx - dx, fy + dy));
            float p21 = get_gray(tex2D<uchar4>(tex, fx,    fy + dy));
            float p22 = get_gray(tex2D<uchar4>(tex, fx + dx, fy + dy));

            float Gx = (p02 + p12 + p22) - (p00 + p10 + p20);
            float Gy = (p20 + p21 + p22) - (p00 + p01 + p02);

            float val = fabsf(Gx) + fabsf(Gy);
```

```

        unsigned char res = (unsigned char)fminf(255.0f, val);

        out[y * w + x] = make_uchar4(res, res, res, 255);
    }
}
}

```

Результаты

1. Отобразить в виде таблички или графиков замеры времени работы ядер с различными конфигурациями (начиная с <<< 1, 32 >>> и как минимум до <<< 1024, 1024 >>>, для ЛР с MPI с различным числом процессов) и различными входными данными (небольшие тесты, средние и предельные). **Обязательно указать единицы измерения времени.**
2. Произвести сравнение с CPU (для этого нужно реализовать свой вариант ЛР без использования технологии CUDA / OpenMP).
3. Если программа подразумевает работу с изображениями (ЛР 2, 3 и 6), то необходимо наличие скриншотов.

Единицы измерения времени в сравнениях - ms.

1. Конфигурация <<< 1, 32 >>>

Image size	time (ms)
736 x 1312	31.5874

2. Конфигурация <<< 256, 256 >>>

Image size	time (ms)
736 x 1312	0.2249

3. Конфигурация <<< 512, 512 >>>

Image size	time (ms)
736 x 1312	0.2232

4. Конфигурация <<< 1024, 1024 >>>

Image size	time (ms)
736 x 1312	0.2634

```

1-2 > G: imageess.cpp > process_cpu(uchar4 *, uchar4 *, int, int)
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <math.h>
4  #include <chrono>
5
6  struct uchar4 {
7      unsigned char x, y, z, w;
8  };
9
10 float get_gray(uchar4 p) {
11     return 0.299f * p.x + 0.587f * p.y + 0.114f * p.z;
12 }
13
14 float get_pixel_mirror(uchar4* data, int w, int h, int x, int y) {
15     if (x < 0) x = -x;
16     if (x >= w) x = 2 * w - x - 1;
17     if (y < 0) y = -y;
18     if (y >= h) y = 2 * h - y - 1;
19     return get_gray(data[y * w + x]);
20 }
21
22 void process_cpu(uchar4* in, uchar4* out, int w, int h) {
23     for (int y = 0; y < h; y++) {
24         for (int x = 0; x < w; x++) {
25
26             float p00 = get_pixel_mirror(in, w, h, x - 1, y - 1);
27             float p01 = get_pixel_mirror(in, w, h, x, y - 1);
28             float p02 = get_pixel_mirror(in, w, h, x + 1, y - 1);
29             float p10 = get_pixel_mirror(in, w, h, x - 1, y);
30             float p12 = get_pixel_mirror(in, w, h, x + 1, y);
31             float p20 = get_pixel_mirror(in, w, h, x - 1, y + 1);
32             float p21 = get_pixel_mirror(in, w, h, x, y + 1);
33             float p22 = get_pixel_mirror(in, w, h, x + 1, y + 1);
34
35             // Оператор Преувеличения
36             float Gx = (p02 + p12 + p22) - (p00 + p10 + p20);
37             float Gy = (p20 + p21 + p22) - (p00 + p01 + p02);
38
39             float val = fabsf(Gx) + fabsf(Gy);
40             if (val > 255.0f) val = 255.0f;
41             unsigned char res = (unsigned char)val;
42
43             out[y * w + x] = {res, res, res, 255};
44         }
45     }
46 }
47
48 int main() {
49     int w, h;
50     FILE *fp = fopen("in.data", "rb");
51     if (!fp) return 1;
52     fread(&w, sizeof(int), 1, fp);
53     fread(&h, sizeof(int), 1, fp);
54     uchar4 *data_in = (uchar4 *)malloc(sizeof(uchar4) * w * h);
55     uchar4 *data_out = (uchar4 *)malloc(sizeof(uchar4) * w * h);
56     fread(data_in, sizeof(uchar4), w * h, fp);
57     fclose(fp);
58
59     // Замер времени
60     auto start = std::chrono::high_resolution_clock::now();
61
62     process_cpu(data_in, data_out, w, h);
63
64     auto end = std::chrono::high_resolution_clock::now();
65     std::chrono::duration<float, std::milli> duration = end - start;
66
67     printf("CPU Image size: %d x %d\n", w, h);
68     printf("CPU Execution time: %.4f ms\n", duration.count());
69
70     fp = fopen("out_cpu.data", "wb");
71     fwrite(&w, sizeof(int), 1, fp);
72     fwrite(&h, sizeof(int), 1, fp);
73     fwrite(data_out, sizeof(uchar4), w * h, fp);

```

Рис. 1 — Реализация ЛР без использования технологии CUDA.

```

snowwy@snowwy-BOM-WXX9:~/Documents/poor_monk/poc
CPU Image size: 736 x 1312
CPU Execution time: 69.0248 ms
snowwy@snowwy-BOM-WXX9:~/Documents/poor_monk/poc

```

Рис. 2 — Замер работы программы на фото-3 до обработки.

```
919191FF B2B2B2FF B2B2B2FF A8A8A8FF
979797FF 7C7C7CFF 515151FF 1F1F1FFF
0D0D0DFF 1A1A1AFF 1A1A1AFF 151515FF
0E0E0EFF 131313FF 181818FF 151515FF
0A0A0AFF 121212FF 0A0A0AFF 1F1F1FFF
333333FF 1E1E1EFF 212121FF 636363FF
777777FF 515151FF 262626FF 454545FF
7A7A7AFF 8A8A8AFF 6E6E6EFF 363636FF
090909FF 212121FF 202020FF 1C1C1CFF
1D1D1DFF 212121FF 1F1F1FFF 111111FF
010101FF 050505FF 030303FF 010101FF
010101FF 000000FF 030303FF 040404FF
-----
[✓] Изображение успешно сохранено в 'out.png'.
snowwy@snowwy-BOM-WXX9:~/Documents/poor_monk/poor_monk/7th_term/pod_mai/parprog-mai/l-2$
```

Рис. 3 — Изображение точно создано после CPU-обработки.



Рис. 4 — Созданное изображение после CPU-обработки.



Рис. 5 — Фото-1 до обработки методом Превитта.



Рис. 6 — Фото-1 после обработки методом Превитта.



Рис. 7 — Фото-2 до обработки методом Превитта.



Рис. 8 — Фото-2 после обработки методом Превитта.



Рис. 9 — Фото-3 до обработки методом Превитта.



Рис. 10 — Фото-3 после обработки методом Превитта.

Выводы

Метод Превитта используется при поиске контуров объектов для их последующей сегментации, при распознавании дорожной разметки или границ дорожного полотна, а также при выделении контуров органов или новообразований на снимках МРТ/КТ.

Основной вызов при разработке на CUDA в управлении иерархией памяти. Необходимость перевода из `uchar4` в `float` требует аккуратности, Использование `cudaTextureObject_t` решило проблему выхода за пределы массива при доступе к пикселям через режим `Mirror`. В режиме дефицита ресурсов (`<<< 1, 32 >>>` — 31.58 мс) GPU работает как крайне медленный процессор. В оптимальном режиме - до 1024, 1024 ускорение относительно первого теста в ~140 раз, а при повышении количества потоков и превышении ими числа пикселей наступает деградация в скорости.